

API Spec

2026-01-21

μEd API - Educational Microservices

Overview

The μEd API ("microservices for education") is a specification for interoperable educational services.

Currently defined endpoints:

- Task evaluation**: automatic feedback and grading for student submissions.
- Chat**: conversational interactions around tasks, submissions, or general learning questions.

Future endpoints (TBD):

- Task/content generation**
- Task/content recommendation**

Version

0.1.0

Contact

μEd API Maintainers

POST /evaluate

Evaluate a submission and generate feedback.

Generates a list of feedback items for a given student submission. The request can optionally include the task context, learner profile, evaluation criteria, pre-submission feedback options, and configuration.

Request Parameters:

Authorization?: **string**; // Optional authorization header.

Request Body:

Content: **application/json** | [EvaluationRequest](#)

```
{
  task?: object,null; // Optional task context that can include prompt content, learning objectives, a
    reference solution, and additional context metadata.
  submission: Submission;
  learnerProfile?: object,null; // Optional learner preferences that can influence tone, detail, and
    language.
  criteria?: array,null; // Optional criteria used for evaluation.
  preSubmissionFeedback?: object,null; // Optional settings for pre-submission feedback (non-
    final). When enabled, the service should avoid committing or finalizing grades.
  configuration?: object,null; // Optional key-value configuration dictionary for provider-specific or
    experimental parameters. Not standardized.
}
```

Example "simpleTextSubmission":

```
{
  "submission": {
    "submissionId": "sub-123",
    "taskId": "task-42",
    "type": "TEXT",
    "content": {
      "text": "Explain what polymorphism is in object-oriented programming."
    },
    "submittedAt": "2025-12-16T09:30:00Z",
    "version": 1
  },
  "configuration": null
}
```

}

Example "preSubmissionFeedbackExample":

```
{
  "submission": {
    "submissionId": "sub-777",
    "taskId": "task-42",
    "type": "TEXT",
    "content": {
      "text": "My short answer..."
    }
  },
  "preSubmissionFeedback": {
    "enabled": true
  },
  "configuration": {
    "llm": {
      "model": "gpt-5.2",
      "temperature": 0.4
    }
  }
}
```

Example "withTaskAndExtras":

```
{
  "task": {
    "taskId": "task-12",
    "title": "Explain polymorphism",
    "content": {
      "text": "Define polymorphism and give at least one example in Java."
    }
  },
  "learningObjectives": [
    "Explain the concept of polymorphism.",
    "Provide an example of subtype polymorphism in Java."
  ],
  "referenceSolution": {
    "text": "Polymorphism allows the same method call to result in different behavior depending on the object's runtime type. For example, a variable of type Shape can reference a Circle or Rectangle, and calling draw() will invoke the appropriate implementation.\n"
  },
  "context": {
    "constraints": "Answer in 3-6 sentences.",
    "language": "en"
  }
},
{
  "submission": {
    "submissionId": "sub-456",
    "taskId": "task-12",
    "type": "TEXT",
    "content": {
      "text": "Polymorphism means that an object can take many forms, for example subclasses implementing methods differently.\n"
    }
  },
  "submittedAt": "2025-12-16T10:00:00Z",
  "version": 2
},
{
  "learnerProfile": {
    "detailPreference": "DETAILED",
    "tonePreference": "FRIENDLY",
    "languagePreference": "en"
  }
},
{
  "criteria": [
    {
      "criterionId": "crit-1",
      "name": "Correctness",
      "context": "The explanation of polymorphism is conceptually correct.",
      "maxPoints": 10
    },
    {
      "criterionId": "crit-2",
      "name": "Clarity",
      "context": "The explanation is clear, well-structured, and easy to understand.",
      "maxPoints": 5
    }
  ]
},
{
  "preSubmissionFeedback": {
    "enabled": false
  },
  "configuration": {
    "llm": {

```

```

    "model": "gpt-5.2",
    "temperature": 0.2,
    "maxTokens": 800,
    "credentials": {
      "apiKey": "OPENAI_API_KEY"
    }
  },
  "enforceRubricStrictness": true
}

```

Response 200:

Successfully generated feedback.

Content: [application/json](#) | [Array<Feedback>](#)

```

{
  feedbackId: string; // Unique identifier of the feedback item.
  title?: string,null; // Optional short label for this feedback item.
  message?: string,null; // Optional feedback text shown to the learner.
  suggestedAction?: string,null; // Optional suggestion for how to act on this feedback.
  awardedPoints?: number,null; // Optional points awarded for this feedback item.
  criterion?: object,null; // Optional criterion linked to this feedback item.
  target?: object,null; // Optional target reference inside the submission.
}

```

Example "exampleResponse":

```

[
  {
    "feedbackId": "fb-1",
    "title": "Clarify your definition",
    "message": "Your explanation of polymorphism is generally correct, but it would help to distinguish between subtype polymorphism and parametric polymorphism.",
    "suggestedAction": "Add one or two concrete examples of polymorphism in Java, e.g., method overriding.",
    "awardedPoints": 2.5,
    "criterion": {
      "criterionId": "crit-1",
      "name": "Correctness",
      "description": "The solution produces correct results for the specified problem.",
      "maxPoints": 10
    },
    "target": {
      "artefactType": "TEXT",
      "locator": {
        "type": "SPAN",
        "startIndex": 0,
        "endIndex": 120
      }
    }
  },
  {
    "feedbackId": "fb-2",
    "title": "Overall structure",
    "message": "The overall structure of your answer is clear and easy to follow.",
    "suggestedAction": null,
    "awardedPoints": null,
    "criterion": null,
    "target": null
  }
]

```

Response 400:

Invalid request (e.g. missing content or invalid schema).

Response 500:

Internal server error.

Response 501:

The server does not support the evaluate method. This allows service providers to implement only subsets of the µEd API.

GET /evaluate/health

Health and capabilities of the evaluation service.

Returns health information and capabilities of the evaluation service. Clients can use this endpoint to discover whether the service supports optional features such as pre-submission feedback, formative feedback, and summative feedback.

Response 200:

Evaluation service is reachable and reporting capabilities.

Content: `application/json` | [EvaluationHealthResponse](#)

```
{
  status: string; // Overall health status of the evaluation service.
  message?: string,null; // Optional human-readable status message.
  version?: string,null; // Optional version of the evaluation service implementation.
  requirements?: object,null; // Optional requirements clients must satisfy to use this service.
  capabilities: EvaluationCapabilities;
}
```

Example "exampleHealth":

```
{
  "status": "ok",
  "message": "Service healthy",
  "version": "1.0.0",
  "capabilities": {
    "supportsEvaluation": true,
    "supportsPreviewEvaluation": false,
    "supportsFormativeFeedback": true,
    "supportsSummativeFeedback": true,
    "supportedArtefactTypes": [
      "TEXT",
      "CODE",
      "MODEL"
    ],
    "supportedLanguages": [
      "en",
      "de"
    ]
  }
}
```

Response 501:

The server does not implement the health endpoint for evaluation.

Response 503:

Evaluation service is currently unavailable or unhealthy.

Content: `application/json` | [EvaluationHealthResponse](#)

```
{
  status: string; // Overall health status of the evaluation service.
  message?: string,null; // Optional human-readable status message.
  version?: string,null; // Optional version of the evaluation service implementation.
  requirements?: object,null; // Optional requirements clients must satisfy to use this service.
  capabilities: EvaluationCapabilities;
}
```

Example "exampleUnhealthy":

```
{
```

```

    "status": "unavailable",
    "message": "Database connection failed",
    "version": "1.0.0",
    "capabilities": {
      "supportsEvaluation": false,
      "supportsPreviewEvaluation": false,
      "supportsFormativeFeedback": false,
      "supportsSummativeFeedback": false,
      "supportedArtefactTypes": [],
      "supportedLanguages": []
    }
  }
}

```

POST /chat

Chat about tasks, submissions, or learning topics.

Conversational endpoint for educational chat use cases. A conversation can be grounded in a specific course, task or submission and may use a learner profile to adapt tone and detail. Typical use cases include: asking follow-up questions on feedback, requesting hints, or clarifying concepts.

Request Parameters:

Authorization?: **string**; // Optional authorization header.

Request Body:

Content: **application/json** | [ChatRequest](#)

```

{
  messages: Array<Message>; // List of messages in the conversation (including history).
  conversationId?: string,null; // Optional identifier for the conversation session.
  learnerProfile?: object,null; // Optional profile to adapt the chat style and level.
  context?: object,null; // Optional educational context (e.g., course material, task context).
  configuration?: object,null; // Optional configuration for the model(s).
}

```

Example "minimalChat":

```

{
  "messages": [
    {
      "role": "user",
      "content": "Can you explain polymorphism?"
    }
  ],
  "configuration": null
}

```

Example "chatWithLlmConfig":

```

{
  "conversationId": "conv-1001",
  "learnerProfile": {
    "detailPreference": "MEDIUM",
    "tonePreference": "FRIENDLY",
    "languagePreference": "en"
  },
  "messages": [
    {
      "role": "user",
      "content": "Give me a hint for my answer about polymorphism."
    }
  ],
  "context": {
    "task": {
      "taskId": "task-12",
      "title": "Explain polymorphism",
      "content": {
        "text": "Define polymorphism and give at least one example in Java."
      }
    }
  }
},

```

```

    "configuration": {
      "llm": {
        "model": "gpt-5.2",
        "temperature": 0.7,
        "stream": false,
        "credentials": {
          "apiKey": "OPENAI_API_KEY"
        }
      }
    }
  }
}

```

Response 200:

Successful chat response.

Content: [application/json](#) | [ChatResponse](#)

```

{
  response: Message; // The generated assistant response.
  metadata?: object,null; // Optional metadata about response generation.
}

```

Response 400:

Invalid request (e.g. missing messages or invalid schema).

Response 500:

Internal server error (e.g. model provider issues).

GET /chat/health

Health and capabilities of the chat service.

Returns health information and capabilities of the chat service. Clients can use this endpoint to discover whether the service supports optional features such as learner profiles or streaming responses.

Response 200:

Chat service is reachable and reporting capabilities.

Content: [application/json](#) | [ChatHealthResponse](#)

```

{
  status: string;
  statusMessage?: string,null; // Optional human-readable status message.
  version?: string,null; // Optional version of the chat service implementation.
  capabilities: ChatCapabilities;
}

```

Example "exampleHealth":

```

{
  "status": "ok",
  "statusMessage": "Service healthy",
  "version": "1.0.0",
  "capabilities": {
    "supportsChat": true,
    "supportsLearnerProfiles": true,
    "supportsStreaming": true,
    "supportedLanguages": [
      "en",
      "de"
    ],
    "supportedModels": [
      "gpt-4o",
      "llama-3"
    ]
  }
}

```

```
}
```

Response 501:

The server does not implement the health endpoint for chat.

Response 503:

Chat service is currently unavailable or unhealthy.

Content: `application/json` | [ChatHealthResponse](#)

```
{
  status: string;
  statusMessage?: string|null; // Optional human-readable status message.
  version?: string|null; // Optional version of the chat service implementation.
  capabilities: ChatCapabilities;
}
```

Example "exampleUnhealthy":

```
{
  "status": "unavailable",
  "statusMessage": "LLM provider connection failed",
  "version": "1.0.0",
  "capabilities": {
    "supportsChat": false,
    "supportsLearnerProfiles": false,
    "supportsStreaming": false,
    "supportedLanguages": [],
    "supportedModels": []
  }
}
```

Schemas

EvaluationRequest

```
{
  task?: object,null; // Optional task context that can include prompt content, learning objectives, a
    reference solution, and additional context metadata.
  submission: Submission;
  learnerProfile?: object,null; // Optional learner preferences that can influence tone, detail, and
    language.
  criteria?: array,null; // Optional criteria used for evaluation.
  preSubmissionFeedback?: object,null; // Optional settings for pre-submission feedback (non-final).
    When enabled, the service should avoid committing or finalizing grades.
  configuration?: object,null; // Optional key-value configuration dictionary for provider-specific or
    experimental parameters. Not standardized.
}
```

ChatRequest

```
{
  messages: Array<Message>; // List of messages in the conversation (including history).
  conversationId?: string,null; // Optional identifier for the conversation session.
  learnerProfile?: object,null; // Optional profile to adapt the chat style and level.
  context?: object,null; // Optional educational context (e.g., course material, task context).
  configuration?: object,null; // Optional configuration for the model(s).
}
```

ChatResponse

```
{
  response: Message; // The generated assistant response.
  metadata?: object,null; // Optional metadata about response generation.
}
```

EvaluationHealthResponse

```
{
  status: string; // Overall health status of the evaluation service.
  message?: string,null; // Optional human-readable status message.
  version?: string,null; // Optional version of the evaluation service implementation.
  requirements?: object,null; // Optional requirements clients must satisfy to use this service.
  capabilities: EvaluationCapabilities;
}
```

EvaluationRequirements

```
{
  requiresAuthorizationHeader: boolean,null; // Optional flag indicating whether an Authorization header
    is required.
  requiresLlmConfiguration: boolean,null; // Optional flag indicating whether configuration.llm must be
    provided.
  requiresLlmCredentials: boolean,null; // Optional flag indicating whether configuration.llm.credentials
```

```

    must be provided.
  }

```

EvaluationCapabilities

```

{
  supportsEvaluation: boolean; // Indicates whether the /evaluate endpoint is implemented and usable.
  supportsPreviewEvaluation: boolean; // Indicates whether the service supports pre-submission
    feedback runs.
  supportsFormativeFeedback: boolean; // Indicates whether the service supports qualitative feedback
    without points.
  supportsSummativeFeedback: boolean; // Indicates whether the service supports feedback with
    points / grading signals.
  supportedArtefactTypes?: array,null; // Optional list of supported artefact types.
  supportedLanguages?: array,null; // Optional list of supported language codes (e.g., 'en', 'de').
}

```

ChatHealthResponse

```

{
  status: string;
  statusMessage?: string,null; // Optional human-readable status message.
  version?: string,null; // Optional version of the chat service implementation.
  capabilities: ChatCapabilities;
}

```

ChatCapabilities

```

{
  supportsChat: boolean; // Indicates whether the /chat endpoint is implemented and usable.
  supportsLearnerProfiles?: boolean; // Indicates whether the service supports adapting to learner
    profiles.
  supportsStreaming?: boolean; // Indicates whether the service supports streaming responses.
  supportedLanguages?: array,null; // Optional list of supported language codes.
  supportedModels?: array,null; // Optional list of supported models.
}

```

LLMConfiguration

```

{
  model: string,null; // Optional model identifier (e.g., 'gpt-4o', 'llama-3').
  temperature: number,null; // Optional sampling temperature.
  maxTokens: integer,null; // Optional maximum number of tokens to generate.
  stream: boolean,null; // Optional flag indicating whether streaming responses are requested.
  credentials: object,null; // Optional credentials object (e.g., API keys, tokens).
}

```

PreSubmissionFeedback

```

{
  enabled: boolean; // Indicates whether pre-submission feedback is requested.
}

```

```
}
```

Task

```
{
  taskId?: string; // Optional unique identifier for the task.
  title: string; // Short title or label for the task.
  content?: object,null; // Optional content shown to the learner (structure is task-specific).
  context?: object,null; // Optional educational context (e.g., course material).
  learningObjectives?: array,null; // Optional list of learning objectives addressed by this task.
  referenceSolution?: object,null; // Optional reference or example solution (structure is task-specific).
  metadata?: object,null; // Optional metadata such as difficulty, topic, tags, etc.
}
```

Submission

```
{
  submissionId?: string; // Optional unique identifier of the submission.
  taskId?: string,null; // Optional identifier of the task this submission belongs to.
  type: ArtefactType;
  content: object; // Logical representation of the submission content.
  submittedAt?: string,null; // Optional timestamp when the submission was created.
  version?: integer,null; // Optional version number (e.g., resubmissions).
}
```

LearnerProfile

```
{
  detailPreference: FeedbackDetailPreference;
  tonePreference: FeedbackTonePreference;
  languagePreference?: string,null; // Optional preferred language code (e.g., 'en', 'de').
  taskProgress?: object,null; // Optional information about the learner's progress on this task/topic.
}
```

Criterion

```
{
  criterionId?: string; // Optional unique identifier of the criterion.
  name: string; // Human-readable name of the criterion.
  context?: string,null; // Optional additional context about how to apply this criterion.
  maxPoints?: number,null; // Optional maximum points for this criterion.
}
```

Feedback

```
{
  feedbackId: string; // Unique identifier of the feedback item.
  title?: string,null; // Optional short label for this feedback item.
  message?: string,null; // Optional feedback text shown to the learner.
  suggestedAction?: string,null; // Optional suggestion for how to act on this feedback.
  awardedPoints?: number,null; // Optional points awarded for this feedback item.
}
```

```

criterion?: object,null; // Optional criterion linked to this feedback item.
target?: object,null; // Optional target reference inside the submission.
}

```

FeedbackTarget

```

{
  artefactType: ArtefactType;
  locator?: object,null; // Optional opaque locator into the submission content. Examples: character
    offsets for text, file + line range for code, element id for models.
}

```

Message

```

{
  role: string;
  content: string;
}

```

ArtefactType

string
 Values: **TEXT, CODE, MODEL, MATH, OTHER**

FeedbackDetailPreference

string
 Values: **BRIEF, MEDIUM, DETAILED**

FeedbackTonePreference

string
 Values: **FORMAL, NEUTRAL, FRIENDLY**